DOCUMENT 16

Developer Guide

Contributing, project structure, adding patterns, and the test suite.

NO GUESSWORK LIGHTING
v1.0 · March 2026 · Confidential

NGW

Who This Guide Is For

This guide is for engineers working on the NGW core engine, frontend, or CLI tooling. It assumes familiarity with Python 3.11+, FastAPI, and React 18. It covers: project layout, backend architecture, how to add or modify a lighting pattern, the test and benchmark systems, and the contribution workflow.

Python 3.11	FastAPI	React 18	Pytest
Backend runtime	REST API layer	Frontend SPA	Test framework

Project Structure

ngw-core — top-level directory layout

```
ngw-core/
■■■ engine/                # Core analysis pipeline
■   ■■■ orchestrator.py    # analyze_image() entry point
■   ■■■ vision_pipeline.py # Image processing + MediaPipe
■   ■■■ vlm_adapter.py     # AI Gateway VLM calls
■   ■■■ solver.py          # Weighted consensus solver
■   ■■■ blueprints/        # Pattern YAML definitions
■   ■■■ learning/          # Failure detection + candidates
■   ■■■ services/          # shoot_match_service.py etc.
■■■ api/
■   ■■■ main.py            # FastAPI app + lifespan
■   ■■■ routes/            # shoot_match.py, lab.py, signals.py …
■■■ db/
■   ■■■ database.py        # init_db(), engine, session
■   ■■■ models.py          # SQLAlchemy table models
■■■ ui/
■   ■■■ src/
■   ■   ■■■ components/    # React components
■   ■   ■■■ pages/         # Route-level pages
■   ■   ■■■ hooks/         # Custom React hooks
■   ■■■ vite.config.ts
■■■ tests/                 # pytest test suite
■■■ scripts/               # CLI tools (see Lab Guide)
■■■ docs/                  # Screenshots, PDFs, markdown
■■■ data/                  # SQLite DB (dev only)
```

Backend Architecture

The FastAPI backend is structured around three layers: routes (HTTP handlers), services (orchestration logic), and the engine (analysis pipeline). Routes are thin — they validate input, call services, and format responses. Services own business logic. The engine is stateless and pure.

Layer	File(s)	Responsibility
Routes	api/routes/*.py	HTTP validation, auth checks, response formatting
Services	engine/services/*.py	Orchestrate engine calls, build response models
Engine	engine/orchestrator.py	analyze_image() — calls pipeline solver blueprint
Pipeline	engine/vision_pipeline.py	Image decode, MediaPipe, region extraction
VLM	engine/vlm_adapter.py	AI Gateway calls, multi-pass, parse responses
Solver	engine/solver.py	Weighted consensus across VLM passes
Blueprints	engine/blueprints/*.yaml	Pattern definitions: tiers, thresholds, copy
DB	db/database.py + db/models.py	SQLAlchemy sessions, table models

Adding a New Lighting Pattern

Patterns are defined in YAML files under engine/blueprints/. The engine loads all YAML files at startup. Adding a pattern requires three steps: write the YAML definition, add gold set images, run benchmarks.

engine/blueprints/loop.yaml — example pattern definition

```yaml
pattern_id:    loop
display_name:  Loop
description:   "Key 45° off-axis, slight elevation. Nose-shadow loops down."

detection:
  confidence_threshold: 0.72
  primary_cues:
    - catchlight_position: [45, 315]   # degrees off center
    - shadow_direction:    loop         # named shadow type
    - fill_ratio_range:    [1.5, 4.0]  # key:fill stop range

blueprint:
  tiers:
    - tier: 1   # exact match
      key:   { type: monolight, modifier: octa_90cm, angle: 45, height: 7.5 }
      fill:  { type: v_flat, side: camera_left }
    - tier: 2   # close substitute
      key:   { type: speedlight, modifier: umbrella_60cm, angle: 45, height: 6.5 }
      fill:  { type: reflector, side: camera_left }

shoot_mode:
  step_order: [camera, key, fill, test, evaluate]
  ceiling_min_ft: 8
```

REQUIRED BLUEPRINT FIELDS

- pattern_id — must be unique, lowercase, underscore-separated.
- confidence_threshold — set conservatively (0.65–0.80). Too high = misses; too low = false positives.
- At least one tier-1 blueprint entry. Tier-5 (ambient) is optional but recommended.
- shoot_mode.ceiling_min_ft — prevents illegal placements on low ceilings.

1 Write the YAML

Create engine/blueprints/{pattern_id}.yaml following the schema above.

2 Validate YAML

Run: python3 scripts/validate_system_yamls.py --strict

3 Add Gold Set images

Add 3–5 labeled reference images via the Lab Gold Sets + Add New.

4 Run benchmarks

Run: python3 scripts/run_benchmarks.py. New pattern shows as NOT_TESTED.

5 Tune thresholds

Adjust confidence_threshold until PASS rate ≥ 75% on gold sets.

6 Write tests

Add test cases to tests/test_engine.py covering detection and blueprint generation.

7 Submit PR

Include: YAML, test results screenshot, benchmark output, gold set IDs.

Adding a New API Endpoint

All API endpoints are FastAPI route functions in api/routes/. Follow the existing patterns: validate with Pydantic, call a service function, return a typed response model.

api/routes/example.py — minimal endpoint pattern

```
from fastapi import APIRouter, Depends, HTTPException
from pydantic import BaseModel
from db.database import get_db
from sqlalchemy.orm import Session

router = APIRouter(prefix="/api/example", tags=["example"])

class ExampleRequest(BaseModel):
    name: str
    value: float

class ExampleResponse(BaseModel):
    result: str
    processed: bool = True

@router.post("/", response_model=ExampleResponse)
async def create_example(
    body: ExampleRequest,
    db: Session = Depends(get_db)
) -> ExampleResponse:
    # Call service layer — never put business logic in routes
    result = example_service.process(db, body.name, body.value)
    return ExampleResponse(result=result)
```

api/main.py — register the router

```
# In api/main.py, import and include the new router:
from api.routes.example import router as example_router

app.include_router(example_router)
# Endpoint is now live at /api/example/
# Visible in Swagger UI at http://localhost:8000/docs
```

Convention	Rule
Route prefix	All routes use /api/ prefix. Lab routes use /api/lab/.
Auth	Lab routes require x-lab-email header checked against LAB_ALLOWED_EMAILS.
Error responses	Raise HTTPException with status_code and detail. Never return raw strings.
Pydantic models	Every request body and response is a typed Pydantic model. No bare dicts.
DB sessions	Always use Depends(get_db). Never import db.engine directly in routes.
Idempotency	POST endpoints that create resources must be idempotent on duplicate IDs.

Frontend Development

The frontend is a React 18 SPA built with Vite 5 and Tailwind CSS 3. The UI proxies all /api/ requests to localhost:8000. Component structure follows a pages components hooks hierarchy.

ui/src — component hierarchy

```
ui/src/
■■■ pages/
■   ■■■ HomePage.tsx          # Entry point — three mode tiles
■   ■■■ AnalyzePage.tsx        # Upload + recommendation display
■   ■■■ ShootModePage.tsx      # Step-by-step on-set instructions
■   ■■■ RecipesPage.tsx        # 13 recipe cards
■   ■■■ MyKitPage.tsx          # Equipment management
■   ■■■ SettingsPage.tsx       # User preferences
■   ■■■ LabPage.tsx            # NGW Lab (restricted)
■■■ components/
■   ■■■ ui/                    # Primitive UI components
■   ■■■ analysis/              # Recommendation display cards
■   ■■■ shoot-mode/            # Step cards, role selector
■   ■■■ lab/                   # Gold set, candidates, signals tabs
■■■ hooks/
    ■■■ useAnalysis.ts         # Upload + poll analysis result
    ■■■ useKit.ts              # My Kit CRUD state
    ■■■ useLabAuth.ts          # Lab email auth state
```

bash — frontend dev commands

```
# Start dev server with HMR
$ cd ui && npm run dev

# Type-check (no emit)
$ cd ui && npm run typecheck

# Lint
$ cd ui && npm run lint

# Production build (outputs to ui/dist/)
$ cd ui && npm run build

# Preview production build locally
$ cd ui && npm run preview
```

Test Suite

The test suite uses pytest with a shared SQLite in-memory database fixture. Tests are organized by domain and must not require a running server or live VLM calls. VLM calls are mocked via pytest-mock.

tests/ — directory layout

```
tests/
■■■ conftest.py                # Shared fixtures: db, client, mock_vlm
■■■ test_engine.py             # Pattern detection + blueprint generation
■■■ test_shoot_match.py        # End-to-end shoot match service
■■■ test_signals.py            # Signal ingestion, hygiene flags
■■■ test_candidates.py         # Candidate CRUD and approval workflow
■■■ test_gold_sets.py          # Gold set CRUD and evaluation
■■■ test_reference_library.py  # Reference library ingestion
■■■ test_perception_layer.py   # Face validation, edge-case flags
■■■ test_learning.py           # Failure detection + severity scoring
```

bash — running tests

```
# Full suite (fastest — ~30 s)
$ python3 -m pytest tests/ -q --tb=short

# Single file with verbose output
$ python3 -m pytest tests/test_engine.py -v

# One specific test by name
$ python3 -m pytest tests/test_signals.py::test_hygiene_flags -v

# With coverage report
$ python3 -m pytest tests/ --cov=engine --cov=api --cov-report=term-missing

# Stop on first failure
$ python3 -m pytest tests/ -x
```

conftest.py — key fixtures

```
@pytest.fixture
def db():
    """In-memory SQLite database, created fresh per test."""
    engine = create_engine("sqlite:///:memory:")
    Base.metadata.create_all(engine)
    with Session(engine) as session:
        yield session

@pytest.fixture
def mock_vlm(mocker):
    """Patch VLM gateway so tests never make real API calls."""
    return mocker.patch(
        "engine.vlm_adapter.gateway.analyze_image",
        return_value=MOCK_VLM_RESPONSE
    )
```

Benchmark System

The benchmark system runs the engine against a curated gold set and reports PASS / SOFT_PASS / FAIL per image. Results gate the CI/CD pipeline — any regression in gold set score below 75% fails the build.

bash — running benchmarks

```
# Standard benchmark run (all patterns)
$ python3 scripts/run_benchmarks.py

# Single pattern only
$ python3 scripts/run_benchmarks.py --pattern loop

# Gold sets only (skip live-data benchmarks)
$ python3 scripts/run_benchmarks.py --gold-sets-only

# JSON output for CI parsing
$ python3 scripts/nightly_benchmark.py --output=json > bench.json

# CI wrapper — exits 1 if any gold set score drops
$ bash scripts/ci_benchmark.sh
```

Result	Meaning	Action Required
PASS	Detected pattern matches gold label	None
SOFT_PASS	Correct family, minor variant difference	Review if count increases
FAIL	Wrong pattern or confidence below threshold	Investigate + file candidate
NOT_TESTED	Pattern has no gold set images yet	Add gold sets via Lab
ERROR	Engine raised an exception on this image	Fix before merging

Contribution Workflow

All contributions follow a branch-per-feature workflow with mandatory tests and benchmark validation before merge. No direct commits to main.

1. **Branch**
   Create a feature branch: git checkout -b feat/your-description

2. **Develop**
   Make changes. Keep commits focused. Run tests frequently with pytest tests/ -q

3. **Test**
   All existing tests must pass. New features need new tests. Coverage should not drop.

4. **Benchmark**
   Run python3 scripts/run_benchmarks.py. No new FAIL results allowed.

5. **Validate YAMLs**
   If blueprints changed: python3 scripts/validate_system_yamls.py --strict

6. **Pull Request**
   Open a PR against main. Include benchmark output and test summary in description.

7 Review

At least one curator review required. High-risk changes (solver, VLM) need two.

8 Merge

Squash-merge after approval. CI re-runs benchmarks as a gate before merge.

WHAT REQUIRES EXTRA REVIEW

- Changes to engine/solver.py — affects all pattern confidence scores.
- Changes to engine/blueprints/*.yaml — affects recommendation output.
- Changes to signal inclusion flags — affects learning and metrics eligibility.
- New VLM prompts — requires before/after benchmark comparison in PR.
- Any change that touches session_signals schema — requires migration script.

Environment Setup for New Developers

New developers need three things: a working Python environment, a populated database, and access to the AI Gateway.

bash — complete new developer setup

```
# 1. Clone and enter the repo
$ git clone https://github.com/your-org/ngw-core.git && cd ngw-core

# 2. Python environment
$ python3 -m venv .venv && source .venv/bin/activate
$ pip install -r requirements.txt

# 3. Frontend
$ cd ui && npm install && cd ..

# 4. Copy and fill in the env file
$ cp .env.example .env.local
# Edit .env.local — fill in AI_GATEWAY_API_KEY and your email

# 5. Initialize and seed the database
$ python3 -c "from db.database import init_db; init_db()"
$ python3 scripts/seed_starter_dataset.py

# 6. Verify everything works
$ python3 scripts/verify_dataset.py
$ python3 -m pytest tests/ -q
$ python3 scripts/run_benchmarks.py

# 7. Start development servers
$ uvicorn main:app --reload &
$ cd ui && npm run dev
```

GETTING LAB ACCESS

- Lab access requires your email in LAB_ALLOWED_EMAILS. Ask your team admin to add it and restart the backend.
- Then in the app: Settings Developer Tools enable, enter your email.
- The Lab tab appears immediately.